DSA Tutorial - 6

Name: Rutuparna Purandare Div: SY1-B1 PRN: B25ET1035

Topic: Implement Stack and Queue using Linked lists in C.

INPUT : STACK USING Linked List-

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
struct Node* top = NULL;
void insert(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) return;
newNode->data = value;
newNode->next = top;
top = newNode;
}
void deleteNode() {
if (top == NULL) {
printf("STACK IS EMPTY\n");
}
else{
struct Node* temp = top;
printf("Deleted value: %d \n",top->data);
top = top->next;
free(temp);}
}
void display() {
struct Node* temp = top;
while (temp != NULL) {
printf("%d \n", temp->data);
temp = temp->next;
}
printf("\n");
}
int main() {
int choice = 0, val;
while (choice != 4) {
printf("CHOOSE\n1. Insert element in stack\n2. Delete an element\n3.
```

```
Display satck\n4. Exit\n");
scanf("%d", &choice);
if (choice == 1) {
printf("Enter value to insert in stack: ");
scanf("%d", &val);
insert(val);
printf("Inserted '%d' in the stack \n",val);
} else if (choice == 2) {
deleteNode();
} else if (choice == 3) {
printf("Current Stack: \n");
display();
} else if (choice == 4) {
printf("Exiting....");
break;
}
}
return 0;
}
```

Queue Using Linked list-

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
struct Node* front = NULL;
struct Node* rear = NULL;
void enqueue(int value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Heap Overflow\n");
return;
}
newNode->data = value;
newNode->next = NULL;
if (rear == NULL) {
front = rear = newNode;
return;
}
rear->next = newNode;
rear = newNode;
}
void dequeue() {
```

```
if (front == NULL) {
printf("QUEUE IS EMPTY\n");
return;
}
struct Node* temp = front;
printf("Deleted value: %d \n", temp->data);
front = front->next;
if (front == NULL) {
rear = NULL;
}
free(temp);
}
void display() {
if (front == NULL) {
printf("QUEUE IS EMPTY\n\n");
return;
}
struct Node* temp = front;
while (temp != NULL) {
printf("%d ", temp->data);
temp = temp->next;
}
printf("\n\n");
}
int main() {
int choice = 0, val;
while (choice != 4) {
printf("CHOOSE\n1. Insert element in queue\n2. Delete an element\n3.
Display queue\n4. Exit\n");
scanf("%d", &choice);
if (choice == 1) {
printf("Enter value to insert in queue: ");
scanf("%d", &val);
enqueue(val);
printf("Inserted '%d' in the queue \n", val);
} else if (choice == 2) {
dequeue();
} else if (choice == 3) {
printf("Current Queue (Front to Rear): \n");
display();
} else if (choice == 4) {
printf("Exiting....");
break;
} else {
```

```
printf("Invalid choice! Try again.\n");
}
}
return 0;
}
```

OUTPUT :

STACK -

Output

```
CHOOSE
1. Insert element in stack
2. Delete an element
3. Display satck
4. Exit
1
Enter value to insert in stack: 10
Inserted '10' in the stack
CHOOSE
1. Insert element in stack
2. Delete an element
3. Display satck
4. Exit
1
Enter value to insert in stack: 20
Inserted '20' in the stack
CHOOSE
1. Insert element in stack
2. Delete an element
3. Display satck
4. Exit
3
Current Stack:
20
10

CHOOSE
1. Insert element in stack
2. Delete an element
3. Display satck
4. Exit
2
Deleted value: 20
```

QUEUE -

Output

```
CHOOSE
1. Insert element in queue
2. Delete an element
3. Display queue
4. Exit
1
Enter value to insert in queue: 20
Inserted '20' in the queue
CHOOSE
1. Insert element in queue
2. Delete an element
3. Display queue
4. Exit
1
Enter value to insert in queue: 30
Inserted '30' in the queue
CHOOSE
1. Insert element in queue
2. Delete an element
3. Display queue
4. Exit
2
Deleted value: 20
CHOOSE
1. Insert element in queue
2. Delete an element
3. Display queue
4. Exit
3
Current Queue (Front to Rear):
30
```